

Concreteness Scoring: Repetitions, Normalization, Text-Level Aggregation, and Word-Sense Disambiguation

Technical note for coauthors

Scope. This note documents exactly how the `lingprops` concreteness scorer treats (1) repeated words, (2) the normalization denominator, (3) sentence-level vs. whole-text accumulation, and (4) word senses under the different WSD strategies. Every numerical value below was produced by running the library; each is reproduced by a closed-form hand check that matches to machine precision.

Code references point to `src/lingprops/concreteness.py` and `src/lingprops/_concreteness_legacy.py`.

1. The unit of accounting: the wordform

Tokenization happens in `legacy.wordformtion(text)` (`_concreteness_legacy.py:397`). It runs `sent_tokenize` -> per-sentence `word_tokenize` -> per-sentence `pos_tag`, but it accumulates every token into a single dictionary for the whole text:

```
wordforms[(word.lower(), tag)] = wordforms.get((word.lower(), tag), 0) + 1 # line 553
```

Consequently the atomic unit everywhere downstream is a **wordform**: the pair (lowercased surface form, POS tag), carrying a frequency **f** counted over the entire text. Sentence boundaries are discarded after this step. This single design choice determines the answers in Sections 3 and 4.

2. The scoring formula

Each POS category (NN, VB, JJ, RB, CD) is scored as an independent partition. For a wordform of depth **d** (number of transitive WordNet hypernyms of its chosen synset) and frequency **f**, the contribution to the partition score is

$$\Delta = \log C(d + f, f)$$

the logarithm of a multiset coefficient (the number of size-**f** multisets drawn from **d**+1 slots). In code: `np.log(comb(depth + 1 + frequency - 1, frequency))` (`concreteness.py:170, :183`), which equals $\log C(d+f, f)$.

A wordform is **valid** (contributes to both numerator and denominator) only if it maps to a WordNet noun-lemma, is not excluded, and has **d** > 0 (the sole exception being the literal lemma "entity"). Words carrying zero concreteness therefore do not inflate the denominator (`concreteness.py:168, :181`).

Two variants

Variant	Frequency used	Per-wordform term	Denominator increment
With repetitions (rep)	actual f	$\log C(d+f, f)$	+ f
Without repetitions (norep)	forced f = 1	$\log C(d+1, 1) = \log(d+1)$	+ 1

rep - `_compute_pos_score` (`concreteness.py:189`): iterates wordforms, uses the real whole-text count **f**, adds **f** to the denominator.

norep - `_compute_pos_score_norep` (`concreteness.py:214`): deduplicates by lemma within the POS partition (WordNetLemmatizer output, before nounification; `concreteness.py:235`), scores each unique lemma once with **f** = 1, adds 1 to the denominator.

Normalization (the denominator)

rep denominator = sum of **f** over valid wordforms in the partition (`concreteness.py:209`).

norep denominator = number of valid unique lemmas in the partition (`concreteness.py:247`).

Final normalized values: $\text{normalized_score} = \text{score} / \text{count}$ and $\text{normalized_score_norep} = \text{score_norep} / \text{count_norep}$ (`concreteness.py:408, :411`).

3. Sentence-level vs. whole-text: repeated words are NOT double-counted

Because of the whole-text aggregation in Section 1, scoring is effectively whole-text, not per-sentence. A concern that would apply to a per-sentence implementation - the same word reappearing in a second sentence contributing a second time and inflating the denominator - does not occur here:

- A noun in three sentences (all NN) becomes one entry ('cat','NN'): 3.
- **rep**: scored once as $\log C(d+3, 3)$; denominator += 3.
- **norep**: deduped to one lemma -> $\log(d+1)$; denominator += 1.

This is demonstrated by the identity of Case 2 and Case 3 in Section 5.

4. Word-sense disambiguation: senses are NOT treated separately

WSD (strategies `first`, `lesk`, `neural`; see `src/lingprops/wsd.py`) only selects which single depth a wordform receives. It never splits a wordform into per-sense entries and never changes the counts:

- The picker is called once per wordform (`concreteness.py:141`) and returns one synset -> one depth.
- Its context is the whole text (`context_tokens = nltk.word_tokenize(text)`, `concreteness.py:365`; the neural picker even caches one context embedding per document).

So a word like `bank` occurring twice with two different senses is still one wordform ('bank','NN'). Its rep denominator counts its frequency; its norep denominator counts it once. Changing the WSD strategy can change the score (a different single depth is chosen) but cannot change either denominator.

Implication. True per-occurrence, per-sentence sense disambiguation is not possible in the current architecture, because `wordformation` discards token positions and sentence membership before scoring. All occurrences of one (word, tag) are indistinguishable and share a single sense/depth.

5. Worked numerical demo

Noun (NN) partition only, `ner=False`. Depths from the real WordNet hierarchy under the default strategy:

word	lemma	depth d
cat	cat	13
dog	dog	14
bank	bank	5

Per-wordform term: $\log C(d+f, f)$; norep term collapses to $\log(d+1)$.

5.1 Scores and denominators

Case	Text	REP score	REP denom	REP norm	NOREP score	NOREP denom	NOREP norm
1 - cat x1	The cat slept.	2.6391	1	2.6391	2.6391	1	2.6391
2 - cat x3, one sentence	The cat saw the cat near the cat.	6.3279	3	2.1093	2.6391	1	2.6391
3 - cat x3, three sentences	The cat slept. A cat ran. My cat purred.	6.3279	3	2.1093	2.6391	1	2.6391
4 - cat + dog, distinct	The cat and the dog played.	5.3471	2	2.6736	5.3471	2	2.6736
5 - cat x3 + dog x1	The cat chased a cat while another cat watched the dog.	9.0360	4	2.2590	5.3471	2	2.6736

Hand checks (all match the library output):

- Case 1: $\log C(14,1) = \log 14 = 2.6391$. No repetition => rep = norep.

- Case 2: $\text{rep log } C(16,3) = 6.3279$, denom 3. $\text{norep log } 14 = 2.6391$, denom 1.
- **Case 3 is bit-identical to Case 2** - the decisive illustration that aggregation is whole-text: three cats across three different sentences leave the score, both denominators, and both normalized values unchanged.
- Case 4: $\text{log } 14 + \text{log } 15 = 5.3471$, denom 2. No repetition $\Rightarrow \text{rep} = \text{norep}$.
- Case 5: $\text{rep log } C(16,3) + \text{log } C(15,1) = 9.0360$, denom $3+1 = 4$. $\text{norep log } 14 + \text{log } 15 = 5.3471$, denom $1+1 = 2$.

5.2 How repetition moves the REP term (fixed depth $d = 13$)

f	REP term $\text{log } C(d+f, f)$	REP normalized (term/f)	NOREP term
1	2.6391	2.6391	2.6391
2	4.6540	2.3270	2.6391
3	6.3279	2.1093	2.6391
5	9.0558	1.8112	2.6391
10	13.9501	1.3950	2.6391
50	29.9794	0.5996	2.6391

The multiset coefficient grows sublinearly in f , while the denominator grows linearly ($+f$). Hence the rep-normalized contribution of a word decreases the more it is repeated - repetition dilutes per-word concreteness. The norep term is flat by construction ($\text{log}(d+1)$), fully immune to repetition.

Limiting cases. $f = 1$: $\text{rep} = \text{norep}$ exactly (Cases 1, 4). $f \rightarrow \text{infinity}$: $\text{log } C(d+f, f) \sim d * \text{log } f$ grows without bound but only logarithmically, so rep-normalized $\rightarrow 0$; norep is unchanged at $\text{log}(d+1)$.

5.3 WSD does not touch the denominators

Text: "I sat by the river bank. Then I went to the bank to deposit cash." (nouns: river, bank x2, cash)

Strategy	REP score	REP denom	NOREP score	NOREP denom
first	6.9157	4	5.6630	3
lesk	6.9157	4	5.6630	3

bank appears twice with two plausible senses but is one wordform. $\text{rep denom} = 1(\text{river}) + 2(\text{bank}) + 1(\text{cash}) = 4$; $\text{norep denom} = 3$ unique lemmas - identical across strategies. Here first and lesk also happened to select the same depth, so even the scores coincide; in general only the score can differ, never the counts.

6. Summary

1. **rep vs. norep.** rep scores each wordform once as $\text{log } C(d+f, f)$ with the true whole-text frequency f and adds f to the denominator; norep deduplicates to unique lemmas per POS, scores each as $\text{log}(d+1)$, and adds 1.
2. **Normalization.** The denominator counts only words with non-zero concreteness - sum of f (rep) or the number of unique valid lemmas (norep).
3. **Text-level, not sentence-level.** Counts are aggregated over the whole text, so a word repeated across sentences is scored once (as a multiset of size f) and never double-counts the numerator or the denominator.
4. **WSD.** Disambiguation acts per wordform using whole-text context and only sets a single depth. Different senses of one surface form are the same unit; WSD changes the score but never the rep/norep denominators.

Reproduce with the accompanying script `demo_repnorep.py`.